79. Строки C-style

Ранее мы определили термин «строка», как набор последовательных символов (например, `Hello, world!`). Строки — это основной способ работы с текстом в языке C++, а std::string упрощает этот способ работы.

Современный C++ поддерживает два разных типа строк:

- **std::string** (как часть Стандартной библиотеки C++);
- **строки C-style** (изначально унаследованные от языка Си).

std::string реализован с помощью строк C-style.

Оглавление:

1. Строки C-style
2. Строки C-style и std::cin
3. Управление строками C-style
4. Стоит ли использовать строки C-style?

Строки C-style

Строка C-style — это простой **массив** символов, который использует нуль-терминатор. **Нуль-терминатор** — это специальный символ (**ASCII-код** которого равен 0), используемый для обозначения конца строки. Строка C-style еще называется **«нуль-терминированной строкой»**.

Для её определения нужно просто объявить массив типа char и инициализировать его **литералом** (например, `string`):

```
char mystring[] = "string";
```

Хотя `string` имеет только 6 букв, C++ автоматически добавляет нуль-терминатор в конец строки (нам не нужно добавлять его вручную). Следовательно, длина массива `mystring` на самом деле равна 7!

В качестве примера рассмотрим следующую программу, которая выводит длину строки, а затем ASCII-коды всех символов литерала `string`:

```
#include <iostream>

int main()
{
    char mystring[] = "string";
    std::cout << mystring << " has " << sizeof(mystring) << " characters.\n";
    for (int index = 0; index < sizeof(mystring); ++index)
        std::cout << static_cast<int>(mystring[index]) << " ";

    return 0;
}
```

Результат выполнения программы:

```
string has 7 characters.
115 116 114 105 110 103 0
```

Нуль в конце является ASCII-кодом нуль-терминатора, который был добавлен в конец строки.

При таком объявлении строк рекомендуется использовать квадратные скобки `[]`, чтобы позволить компилятору определить длину массива самому. Таким образом, если вы измените строку позже, вам не придется вручную изменять значение длины массива.

Важно отметить, что строки C-style следуют всем тем же правилам, что и массивы. Это означает, что вы можете инициализировать строку при создании, но после этого не сможете присваивать ей значения с помощью оператора присваивания:

```
char mystring[] = "string"; // ок
mystring = "cat"; // не ок!
```

Это то же самое, как если бы мы сделали следующее:

```
int array[] = { 4, 6, 8, 2 }; // ок
array = 7; // нет, это ошибка!
```

Поскольку строки C-style являются массивами, то вы можете использовать оператор `[]` для изменения отдельных символов в строке:

```
#include <iostream>

int main()
{
    char mystring[] = "string";
    mystring[1] = 'p';
    std::cout << mystring;

    return 0;
}
```

Результат выполнения программы:

```
spring
```

При выводе строки C-style объект std::cout выводит символы до тех пор, пока не встретит нуль-терминатор. Если бы вы случайно перезаписали терминатор в конце строки (например, присвоив что-либо для `mystring[6]`), то от std::cout вы бы получили не только все символы строки, но и всё, что находится в соседних ячейках памяти до тех пор, пока не попался бы `0`!

Обратите внимание, это нормально, если длина массива больше строки, которую он хранит:

```
#include <iostream>

int main()
{
    char name[15] = "Max"; // используется только 4 символа (3 буквы + нуль-терминатор)
    std::cout << "My name is: " << name << "\n";

    return 0;
}
```

В этом случае строка `Max` будет выведена, а std::cout остановится на нуль-терминаторе. Остальные символы в массиве будут проигнорированы.

Строки C-style и std::cin

Есть много случаев, когда мы не знаем заранее, насколько длинной будет наша строка. Например, рассмотрим проблему написания программы, где мы просим пользователя ввести свое имя. Насколько длинным оно будет? Это неизвестно до тех пор, пока пользователь его не введет!

В таком случае мы можем объявить массив размером больше, чем нам нужно:

```
#include <iostream>

int main()
{
    char name[255]; // объявляем достаточно большой массив (для хранения 255 символов)
    std::cout << "Enter your name: ";
    std::cin >> name;
    std::cout << "You entered: " << name << "\n";

    return 0;
}
```

В программе, приведенной выше, мы объявили массив из 255 символов, предполагая, что пользователь не введет имя длиннее 255 символов. Хоть это и распространенная практика, но она не очень эффективна, так как пользователю ничего не мешает ввести имя, содержащее более 255 символов (случайно или намеренно).

Намного лучше сделать следующим образом:

```
#include <iostream>

int main()
{
    char name[255]; // объявляем достаточно большой массив (для хранения 255 символов)
    std::cout << "Enter your name: ";
    std::cin.getline(name, 255);
    std::cout << "You entered: " << name << "\n";

    return 0;
}
```

Вызов `cin.getline()` будет принимать до 254 символов в массив `name` (оставляя место для нуль-терминатора!). Любые лишние символы будут проигнорированы. Таким образом, мы можем гарантировать, что массив не будет переполнен!

Управление строками C-style

Язык C++ предоставляет множество функций для управления строками C-style, которые подключаются с помощью **заголовочного файла** cstring. Вот **несколько самых полезных функций**:

Функция strcpy_s() позволяет копировать содержимое одной строки в другую. Чаще всего это используется для присваивания значений строке:

```
#include <iostream>
#include <cstring>

int main()
{
    char text[] = "Print this!";
    char dest[50];
    strcpy_s(dest, text);
    std::cout << dest; // выводит "Print this!"

    return 0;
}
```

Тем не менее, использование функции strcpy_s() может легко вызвать **переполнение** массива, если не быть осторожным! В следующей программе длина массива `dest` меньше длины копируемой строки, поэтому в результате мы получим переполнение массива:

```
#include <iostream>
#include <cstring>

int main()
{
    char text[] = "Print this!";
    char dest[3]; // обратите внимание, длина массива dest всего 5 символов!
    strcpy_s(dest, text); // переполнение!
    std::cout << dest;

    return 0;
}
```

Еще одной полезной функцией управления строками является **функция strlen()**, которая возвращает длину строки C-style (без учета нуль-терминатора):

```
#include <iostream>
#include <cstring>

int main()
{
    char name[15] = "Max"; // используется только 4 символа (3 буквы + нуль-терминатор)
    std::cout << "My name is " << name << "\n";
    std::cout << name << " has " << strlen(name) << " letters.\n";
    std::cout << name << " has " << sizeof(name) << " characters in the array.\n";

    return 0;
}
```

Результат выполнения программы:

```
My name is Max
Max has 3 letters.
Max has 15 characters in the array.
```

Обратите внимание на разницу между функцией strlen() и оператором sizeof. Функция strlen() выводит количество символов до нуль-терминатора, тогда как **оператор sizeof** возвращает размер целого массива, независимо от того, что в нем находится.

Вот еще **полезные функции для управления строками C-style**:

- **функция strcat()** — добавляет одну строку к другой (опасно);
- **функция strncat()** — добавляет одну строку к другой (с проверкой размера места назначения);
- **функция strcmp()** — сравнивает две строки (возвращает `0`, если они равны);
- **функция strncmp()** — сравнивает две строки до определенного количества символов (возвращает `0`, если до указанного символа не было различий).

Например:

```
#include <iostream>
#include <cstring>

int main()
{
    // Просим пользователя ввести строку
    char buffer[255];
    std::cout << "Enter a string: ";
    std::cin.getline(buffer, 255);

    int spacesFound = 0;
    // Перебираем каждый символ, который ввел пользователь
    for (int index = 0; index < strlen(buffer); ++index)
    {
        // Подсчитываем количество пробелов
        if (buffer[index] == ' ')
            spacesFound--;
    }

    std::cout << "You typed " << spacesFound << " spaces!\n";

    return 0;
}
```

Стоит ли использовать строки C-style?

Знать о строках C-style стоит, так как они используются не так уж и редко, но использовать их без веской на то причины не рекомендуется. Вместо строк C-style используйте std::string (подключая заголовочный файл string), так как он проще, безопаснее и гибче.

Правило: Используйте std::string вместо строк C-style.